

```

enter main()
  enter readArray()
  leave readArray()
  enter quicksort(1,9)
    enter partition(1,9)
    leave partition(1,9)
    enter quicksort(1,3)
      ...
    leave quicksort(1,3)
    enter quicksort(5,9)
      ...
    leave quicksort(5,9)
  leave quicksort(1,9)
leave main()

```

Figure 7.3: Possible activations for the program of Fig. 7.2

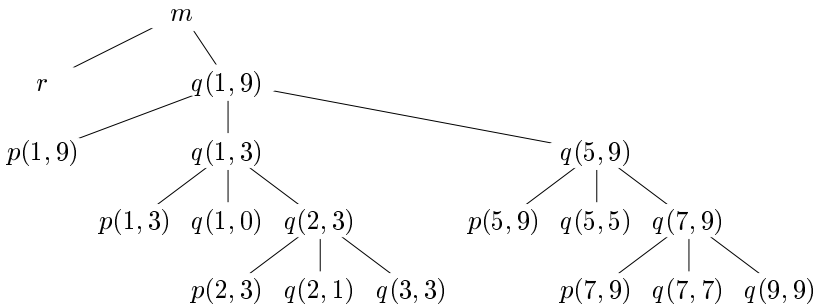


Figure 7.4: Activation tree representing calls during an execution of *quicksort*

7.2.2 Activation Records

Procedure calls and returns are usually managed by a run-time stack called the *control stack*. Each live activation has an *activation record* (sometimes called a *frame*) on the control stack, with the root of the activation tree at the bottom, and the entire sequence of activation records on the stack corresponding to the path in the activation tree to the activation where control currently resides. The latter activation has its record at the top of the stack.

Example 7.3: If control is currently in the activation $q(2,3)$ of the tree of Fig. 7.4, then the activation record for $q(2,3)$ is at the top of the control stack. Just below is the activation record for $q(1,3)$, the parent of $q(2,3)$ in the tree. Below that is the activation record $q(1,9)$, and at the bottom is the activation record for m , the main function and root of the activation tree. $\square$

We shall conventionally draw control stacks with the bottom of the stack higher than the top, so the elements in an activation record that appear lowest on the page are actually closest to the top of the stack.

The contents of activation records vary with the language being implemented. Here is a list of the kinds of data that might appear in an activation record (see Fig. 7.5 for a summary and possible order for these elements):

Actual parameters
Returned values
Control link
Access link
Saved machine status
Local data
Temporaries

Figure 7.5: A general activation record

1. Temporary values, such as those arising from the evaluation of expressions, in cases where those temporaries cannot be held in registers.
2. Local data belonging to the procedure whose activation record this is.
3. A saved machine status, with information about the state of the machine just before the call to the procedure. This information typically includes the *return address* (value of the program counter, to which the called procedure must return) and the contents of registers that were used by the calling procedure and that must be restored when the return occurs.
4. An “access link” may be needed to locate data needed by the called procedure but found elsewhere, e.g., in another activation record. Access links are discussed in Section 7.3.5.
5. A *control link*, pointing to the activation record of the caller.
6. Space for the return value of the called function, if any. Again, not all called procedures return a value, and if one does, we may prefer to place that value in a register for efficiency.
7. The actual parameters used by the calling procedure. Commonly, these values are not placed in the activation record but rather in registers, when possible, for greater efficiency. However, we show a space for them to be completely general.

Example 7.4: Figure 7.6 shows snapshots of the run-time stack as control flows through the activation tree of Fig. 7.4. Dashed lines in the partial trees go to activations that have ended. Since array *a* is global, space is allocated for it before execution begins with an activation of procedure *main*, as shown in Fig. 7.6(a).

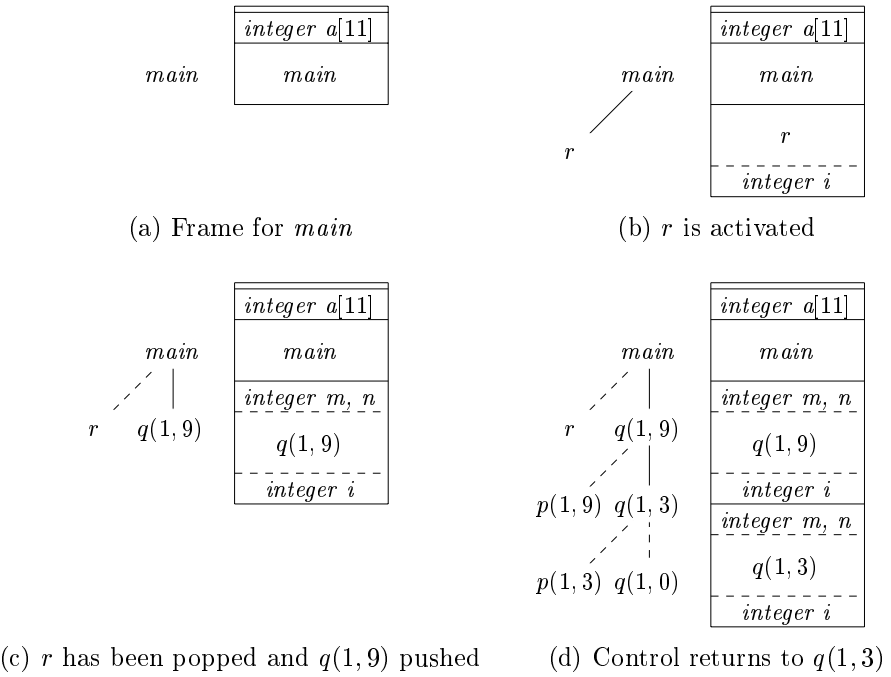


Figure 7.6: Downward-growing stack of activation records

When control reaches the first call in the body of *main*, procedure *r* is activated, and its activation record is pushed onto the stack (Fig. 7.6(b)). The activation record for *r* contains space for local variable *i*. Recall that the top of stack is at the bottom of diagrams. When control returns from this activation, its record is popped, leaving just the record for *main* on the stack.

Control then reaches the call to *q* (quicksort) with actual parameters 1 and 9, and an activation record for this call is placed on the top of the stack, as in Fig. 7.6(c). The activation record for *q* contains space for the parameters *m* and *n* and the local variable *i*, following the general layout in Fig. 7.5. Notice that space once used by the call of *r* is reused on the stack. No trace of data local to *r* will be available to *q(1, 9)*. When *q(1, 9)* returns, the stack again has only the activation record for *main*.

Several activations occur between the last two snapshots in Fig. 7.6. A recursive call to *q(1, 3)* was made. Activations *p(1, 3)* and *q(1, 0)* have begun and ended during the lifetime of *q(1, 3)*, leaving the activation record for *q(1, 3)*